



**Assessment Title: Continuous
Assessment2
System Modelling & Build**

Driss Bzioui

Module Title:	Algorithms & Constructs
Assessment Title:	CA2 - System Modelling & Build
Lecturer Name:	Mr. Taufique Ahmed
Student Full Name:	Driss Bzioui
Student Number:	2025488
Assessment Issue Date:	Tuesday 21st April
Date of Submission:	Saturday 23 May

Table of Contents

ALGORITHM JUSTIFICATION REPORT	4
1. Sorting Algorithm Choice: Merge Sort.....	4
2. Searching Algorithm Choice: Recursive Binary Search.....	5
GitHub Repository	5

ALGORITHM JUSTIFICATION REPORT

Organisation Chosen: Bank

1. Sorting Algorithm Choice: Merge Sort

I used merge sort for sorting the names from the file. I picked this one because its recursive which is what the assignment asked for.

The reason I went with merge sort over quick sort is because in class we saw that quick sort can be slow if the data is already in some kind of order. Since the names in the file could be in any order I wanted something that works the same speed no matter what. Merge sort does that.

I also thought about bubble sort because its the easiest one to code but its way too slow when theres lots of names. Like it has to go through the whole list over and over. With merge sort it splits everything in half so it finishes much quicker.

In my code the sort method takes the list, splits it into two halves, sorts each half by calling itself again (thats the recursion), then merges them back together in alphabetical order. I used `compareToIgnoreCase` so it doesnt matter if names have capitals or not.

I tested it with the 45 names from the file and it sorts them instantly. If the bank had way more employees it would still be fast because splitting in half is efficient.

2. Searching Algorithm Choice: Recursive Binary Search

For searching I used binary search because its fast and recursive.

The way it works in my code is it looks at the middle of the list and checks if thats the name. If the name comes before it alphabetically it looks in the first half, if after it looks in the second half. Then it does the same thing again on that smaller part. The method keeps calling itself with a smaller range until it finds the name or runs out of places to look.

I needed the list to be sorted first for this to work. So in my searchList() method I check if sorted is true and if not I sort it first before searching. That way it always works.

I could have just used a for loop to go through every name one by one (linear search) but that would be slower especially if the name is near the end. Binary search is quicker because it skips half the list each time.

The recursive part is the doSearch method calling itself with new lo and hi values. When lo goes past hi it means the name wasnt found so it returns -1. I also used compareToIgnoreCase so searching for "john smith" finds "John Smith".

GitHub Repository:

<https://github.com/dbzioui22-ux/Bank-System-CA2.git>